

Roteiro de Apresentação — BigZ: Surfando em Hábitos Sustentáveis

T300 Programação Funcional — UNIFOR | Prof. Bruno Lopes Duração estimada: 28–30 minutos

Estrutura do vídeo

Bloco	Responsável	Tempo estimado
Abertura e contexto	Equipe (todos)	~3 min
Módulo A — Autenticação e Perfil	João Pedro	~7 min
Módulo B — Gestão de Hábitos	Luiz Carlos	~7 min
Módulo C — Registro e Acompanhamento	Ricardo André	~8 min
Requisitos Técnicos (RT01–RT07)	Equipe (todos)	~4 min
Encerramento	Equipe	~1 min

BLOCO 1 — Abertura e Contexto (~3 min) — Equipe

Fala sugerida

"Olá, professor Bruno. Somos a equipe do projeto BigZ — Surfando em Hábitos Sustentáveis. A equipe é composta por João Pedro Mendes Moreira, Luiz Carlos e Ricardo André.

O BigZ é uma aplicação web colaborativa desenvolvida em Elixir com Phoenix LiveView, onde usuários registram e acompanham hábitos sustentáveis do cotidiano — como reduzir o uso de plástico, economizar água ou optar por transporte ativo — acumulando pontos e visualizando o engajamento da comunidade em tempo real.

As tecnologias utilizadas foram:

- **Elixir** com **Phoenix Framework** e **Phoenix LiveView** para a aplicação web reativa
- **Ecto** + **PostgreSQL** (hospedado no Supabase) para persistência
- **HEEx** + **Tailwind CSS** com DaisyUI para a interface
- **Phoenix.PubSub** para atualizações em tempo real
- **mix phx.gen.auth** para o sistema de autenticação

A divisão foi: João Pedro ficou com o Módulo A (Autenticação e Perfil), Luiz com o Módulo B (Gestão de Hábitos) e Ricardo com o Módulo C (Registro e Acompanhamento)."

[TELA: Mostrar a tela inicial do BigZ rodando no navegador — página de login]

BLOCO 2 — Módulo A: Autenticação e Perfil (~7 min) — João Pedro

RF01 — Cadastro de usuário (~2 min)

[TELA: Abrir `/users/register`]

Fala:

"O RF01 exige cadastro com nome, e-mail e senha usando `mix phx.gen.auth`. O gerador criou toda a estrutura de autenticação. Eu adicionei o campo `name` ao schema de usuário e ao formulário de cadastro.

Vou cadastrar um usuário agora..."

[AÇÃO: Preencher o formulário com nome, e-mail e senha e submeter]

Explicar no código:

- Arquivo `lib/bigz/accounts/user.ex` — changeset de registro com validações de nome, e-mail, senha
 - Campo `name` adicionado via migration
 - Validação de formato de e-mail e tamanho mínimo de senha
-

RF02 — Login e Logout (~2 min)

[TELA: Navegar para `/users/log-in`]

Fala:

"O RF02 cobre login e logout com sessão persistente. O Phoenix `gen.auth` gerou o `UserController` que gerencia tokens de sessão. Há também suporte a 'remember me' para manter a sessão entre fechamentos do navegador.

Vou fazer login agora..."

[AÇÃO: Fazer login com o usuário cadastrado] [AÇÃO: Mostrar o logout clicando em "Sair" no menu do usuário] [AÇÃO: Fazer login novamente para continuar]

Explicar no código:

- `lib/bigz_web/user_auth.ex` — `log_in_user/3`, cookie de remember me, `log_out_user/1`
 - Token de sessão armazenado em banco (`users_tokens`)
-

RF03 — Página de perfil (~3 min)

[TELA: Navegar para `/profile`]

Fala:

"O RF03 exige uma página de perfil com nome, bio editável e pontuação total acumulada. A pontuação não vem de um campo direto no banco — ela é calculada em tempo real através de uma query que soma os pontos dos check-ins do usuário via JOIN com a tabela de hábitos."

[AÇÃO: Mostrar o perfil com nome, bio e pontuação] [AÇÃO: Editar a bio e salvar]

Explicar no código:

- `lib/bigz_web/live/user_live/profile.ex` — mount carrega `total_points` via `Habits.sum_user_points(scope)`
- `Habits.sum_user_points/1` em `lib/bigz/habits.ex` — JOIN entre checkins e habits, sem atualizar o campo `score` diretamente

BLOCO 3 — Módulo B: Gestão de Hábitos (~7 min) — Luiz Carlos

RF04 — Cadastro de hábitos (~2 min)

[TELA: Navegar para `/habits/new`]

Fala:

"O RF04 exige cadastro de hábitos com nome, descrição, categoria e pontuação. As categorias permitidas são: alimentação, transporte, energia, água e resíduos. A pontuação é validada para ser positiva."

[AÇÃO: Criar um novo hábito preenchendo todos os campos]

Explicar no código:

- `lib/bigz/habits/habit.ex` — changeset com `validate_inclusion` para categoria e `validate_number` para pontuação
- `lib/bigz_web/live/habit_live/index.ex` — LiveView com `handle_event "save"`

RF05 — Listagem com filtro por categoria (~2 min)

[TELA: Navegar para `/habits`]

Fala:

"O RF05 exige listagem de hábitos com filtro por categoria. Qualquer usuário, mesmo sem login, pode ver a lista pública de hábitos. O filtro é feito via parâmetros de URL e aplicado diretamente na query Ecto."

[AÇÃO: Mostrar a lista de hábitos] [AÇÃO: Usar o filtro de categoria — clicar em "Energia", "Transporte", etc.]

Explicar no código:

- `Habits.list_habits/2` — `case Map.get(filters, "category")` filtra via Ecto query
- `handle_params` no LiveView atualiza o filtro sem recarregar a página

RF06 — Edição e remoção (~3 min)

[TELA: Na lista de hábitos, encontrar um hábito criado pelo usuário logado]

Fala:

"O RF06 exige edição e remoção, mas apenas pelo próprio criador do hábito. A autorização é feita no contexto: compara o `user_id` do hábito com o `user_id` da sessão atual. Se não for o dono, retorna `{:error, :unauthorized}`."

[AÇÃO: Clicar em Editar — modificar o nome e salvar] [AÇÃO: Tentar acessar a edição de um hábito de outro usuário (se houver) para mostrar a proteção]

Explicar no código:

- `Habits.update_habit/3` e `Habits.delete_habit/2` — verificação de `habit.user_id == current_scope.user.id`
- Botões de editar/deletar só aparecem para o dono (`:if` no template HEx)

BLOCO 4 — Módulo C: Registro e Acompanhamento (~8 min) — Ricardo André

RF07 — Check-in diário com proteção contra duplicidade (~3 min)

[TELA: Navegar para `/habits`, clicar em "Registrar Check-in" em um hábito]

Fala:

"O RF07 exige registro diário de hábitos com impedimento de duplicidade no mesmo dia para o mesmo usuário. Vou demonstrar o check-in e depois tentar registrar novamente o mesmo hábito hoje."

[AÇÃO: Registrar um check-in com sucesso — mostrar a mensagem "Check-in registrado!"] [AÇÃO: Tentar registrar o mesmo hábito no mesmo dia — mostrar "Você já registrou este hábito hoje."]

Explicar no código:

```
lib/bigz/habits.ex – create_checkin/2:  
- user_id vem de current_scope.user.id (definido pela sessão, nunca pelo usuário)  
- checkin_date: Date.utc_today() (definido no servidor, não aceita data do cliente)  
- Repo.insert() dispara o unique_constraint do banco
```

```
lib/bigz/habits/checkin.ex – changeset:  
- unique_constraint(:user_id, name: :checkins_user_id_habit_id_checkin_date_index)  
- message: "Você já registrou este hábito hoje."
```

```
lib/bigz_web/live/checkin_live/new.ex:  
- Rota protegida: require_authenticated_user  
- handle_event "save" chama Habits.create_checkin(scope, habit)
```

RF08 — Dashboard pessoal (~3 min)

[TELA: Navegar para [/inicio](#)]

Fala:

"O RF08 exige um dashboard pessoal com histórico de check-ins e pontuação acumulada por semana. O dashboard mostra quatro métricas: pontuação total, pontuação desta semana, check-ins desta semana e número de hábitos.

O gráfico de barras mostra as últimas 6 semanas ISO — segunda a domingo — com a pontuação acumulada em cada semana. O histórico lista os últimos check-ins com nome do hábito, categoria, pontos e data.

Um detalhe importante: toda a pontuação é calculada via JOIN entre check-ins e hábitos. O campo `score` da tabela de usuários existe mas nunca é atualizado — evitamos duas fontes de verdade no banco."

[AÇÃO: Mostrar o dashboard com estatísticas, barras semanais e histórico] [AÇÃO: Fazer mais um check-in e voltar ao dashboard para mostrar atualização]

Explicar no código:

```
lib/bigz/habits.ex:
```

- `sum_user_points/1` → `SELECT SUM(h.points) JOIN habits WHERE user_id = ?`
- `sum_user_points_this_week/1` → filtra pela semana atual (segunda a domingo UTC)
- `list_weekly_summaries/2` → agrupa check-ins por semana em Elixir (sem `date_trunc`)
- `list_user_checkins/2` → JOIN preload, sem N+1

```
lib/bigz_web/live/home_live/index.ex:
```

- `mount` carrega todas as métricas de uma só vez
- `format_week_label/1` exibe "Semana atual" para a semana corrente
- `bar_pct/2` calcula o percentual para a barra de progresso

RF09 — Feed da comunidade em tempo real (~2 min)

[TELA: Abrir duas janelas do navegador lado a lado — [/comunidade](#) nas duas]

Fala:

"O RF09 exige um feed global em tempo real com Phoenix.PubSub. Vou abrir duas janelas na página da comunidade. Quando eu registrar um check-in em outra aba, ele deve aparecer instantaneamente aqui, sem nenhum refresh.

O feed mostra: nome do usuário, nome do hábito, categoria, pontos e horário. O e-mail nunca é exibido para proteger a privacidade."

[AÇÃO: Em uma janela, ir em `/habits` e registrar um check-in] [AÇÃO: Mostrar o check-in aparecendo instantaneamente na outra janela aberta em `/comunidade`]

Explicar no código:

```
lib/bigz_web/live/community_live/index.ex:  
- mount: if connected?(socket) → PubSub.subscribe(topic)  
  (só assina quando o WebSocket está estabelecido, não no HTTP inicial)  
- handle_info({:new_checkin, checkin}, socket) → stream_insert(socket, :checkins,  
  checkin, at: 0)  
  (Phoenix streams: deduplication por DOM id — mesmo check-in nunca aparece duas  
  vezes)  
- display_name/1 e user_initials/1 usam apenas user.name, nunca o e-mail
```

```
lib/bigz/habits.ex — create_checkin/2:  
- Após Repo.insert com sucesso, preloada user + habit  
- Phoenix.PubSub.broadcast(Bigz.PubSub, "checkins:community", {:new_checkin,  
  checkin})  
  (broadcast só ocorre após confirmação do banco)
```

BLOCO 5 — Requisitos Técnicos Obrigatórios (~4 min) — Equipe

[TELA: Alternar entre código e aplicação conforme cada RT]

RT01 — Phoenix com LiveView

"Toda a interface é construída com Phoenix LiveView. Não há páginas estáticas — cada rota usa `live/3` no router."

[TELA: Mostrar `lib/bigz_web/router.ex` — linhas `live "/inicio"`, `live "/habits"`, etc.]

RT02 — Banco de dados via Ecto (PostgreSQL)

"Usamos PostgreSQL hospedado no Supabase. Todo o acesso ao banco é via Ecto: queries com `from`, `join`, `where`, `preload` — sem SQL raw."

[TELA: Mostrar `lib/bigz/habits.ex` — função `list_community_checkins/1` com o JOIN]

RT03 — Autenticação via mix `phx.gen.auth`

"Rodamos `mix phx.gen.auth` para gerar toda a estrutura de autenticação: tokens de sessão, magic link, sudo mode, plugs de proteção de rota."

[TELA: Mostrar `lib/bigz_web/user_auth.ex` — `fetch_current_scope_for_user` e `require_authenticated_user`]

RT04 — Layout com HEEEx + Tailwind CSS

"Os templates usam HEEEx — o HTML com interpolação de Elixir do Phoenix. O estilo é todo em Tailwind CSS com DaisyUI, sem CSS customizado."

[TELA: Mostrar `lib/bigz_web/components/layouts.ex` — a estrutura do componente `app/1`]

RT05 — LiveView com atualização em tempo real via Phoenix.PubSub

"O `CommunityLive.Index` subscreve ao tópico `checkins:community` quando o WebSocket conecta, e o `handle_info` atualiza o stream sem refresh. Acabamos de demonstrar isso."

[TELA: Mostrar as 5 linhas-chave do `community_live/index.ex`: `subscribe` + `handle_info`]

RT06 — Changesets com validações no Ecto

"Hábitos validam: nome obrigatório, categoria dentro do enum, pontuação positiva. Check-ins validam: `unique_constraint` de usuário + hábito + data. Usuários validam: formato de e-mail, tamanho mínimo de senha."

[TELA: Mostrar `lib/bigz/habits/habit.ex` — `changeset` com `validate_required`, `validate_inclusion`, `validate_number`]

RT07 — Navegação via router do Phoenix

"Todas as rotas são declaradas no router Phoenix. Rotas autenticadas ficam dentro de `live_session :require_authenticated_user` com `on_mount: require_authenticated`. Rotas públicas ficam em `live_session :current_user`."

[TELA: Mostrar `lib/bigz_web/router.ex` — as duas `live_sessions`]

BLOCO 6 — Encerramento (~1 min) — Equipe

Fala:

"Esse foi o BigZ — Surfando em Hábitos Sustentáveis. Conseguimos implementar todos os requisitos técnicos (RT01–RT07) e todos os requisitos funcionais dos três módulos (RF01–RF09). Obrigado professor!"

Checklist pré-gravação

Antes de gravar, verificar:

- ☐ Aplicação rodando localmente (`mix phx.server`)
- ☐ Banco de dados conectado (Supabase acessível)
- ☐ Usuário(s) de teste criados previamente para agilizar a demo
- ☐ Pelo menos 2 usuários com hábitos e check-ins para mostrar o feed da comunidade

- ☐ Duas janelas do navegador abertas para demonstrar o PubSub (RF09)
- ☐ Editor de código aberto nos arquivos-chave para as explicações técnicas
- ☐ Resolução da tela adequada — testar captura de tela antes de gravar
- ☐ Microfone testado
- ☐ Sem notificações na tela durante a gravação

Arquivos-chave para ter abertos durante a gravação

Módulo	Arquivo principal	Arquivo de suporte
A	<code>lib/bigz/accounts/user.ex</code>	<code>lib/bigz_web/live/user_live/profile.ex</code>
B	<code>lib/bigz/habits/habit.ex</code>	<code>lib/bigz_web/live/habit_live/index.ex</code>
C	<code>lib/bigz/habits.ex</code>	<code>lib/bigz_web/live/community_live/index.ex</code>
RTs	<code>lib/bigz_web/router.ex</code>	<code>lib/bigz_web/user_auth.ex</code>